

**LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL VIII
ABSTRACT & INTERFACE**



Disusun Oleh:

Arif Mufti Tharsa 105224041

**PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA
2026**

I. Pendahuluan

Di THT Modul 8 ini, saya membuat program Java yang terdiri dari satu interface, satu abstract class, dan tiga class konkret dalam satu file, yaitu interface AksiBertarung, abstract class Karakter, class Pahlawan, class Monster, dan class GameEngine sebagai entry point. Program ini mensimulasikan permainan dungeon berbasis teks di mana pemain mengendalikan seorang Pahlawan untuk menghadapi tiga Monster secara berurutan. Materi utama yang diterapkan adalah Polymorphism, yaitu kemampuan satu method untuk memiliki banyak bentuk implementasi yang berbeda tergantung pada class yang menggunakannya, yang diwujudkan melalui method overriding dan method overloading.

Secara umum, program ini dirancang agar setiap jenis karakter dapat merespons aksi bertarung dengan cara yang berbeda meski berasal dari hierarki class yang sama. Interface AksiBertarung mendefinisikan kontrak tiga method wajib: `serang()`, `bertahan()`, dan `gunakanItem()`. Abstract class Karakter mengimplementasikan interface tersebut secara parsial dan menambahkan method abstract `tampilkanStatus()` yang wajib di-override oleh subclass. Class Pahlawan juga menerapkan method overloading pada `serang()` dengan menyediakan dua versi berbeda parameter. Semua interaksi antarkarakter dikelola oleh GameEngine menggunakan loop pertarungan berbasis input pemain.

II. Variabel

No	Nama Variabel	Tipe Data	Fungsi
1	nama	String	Menyimpan nama karakter yang digunakan baik oleh Pahlawan maupun Monster (atribut Karakter)
2	hp	int	Menyimpan poin kesehatan karakter; jika mencapai 0 karakter dinyatakan kalah (atribut Karakter)
3	baseDamage	int	Menyimpan nilai serangan dasar yang menjadi acuan perhitungan damage (atribut Karakter)
4	isDefending	boolean	Menandai apakah karakter sedang dalam mode bertahan; jika true, damage yang diterima dipotong 50% (atribut Karakter)
5	mana	int	Menyimpan poin mana Pahlawan yang dikonsumsi saat menggunakan skill (atribut Pahlawan)
6	level	int	Menyimpan level Pahlawan sebagai pengali dalam perhitungan damage serangan (atribut Pahlawan)
7	jenisMonster	String	Menyimpan tipe atau jenis Monster: Normal, Brute, atau Boss (atribut Monster)
8	hero	Pahlawan	Objek Pahlawan yang dikendalikan pemain dan menjadi karakter utama dalam GameEngine
9	daftarMonster	Monster[]	Array yang menyimpan tiga objek Monster yang akan dihadapi pemain secara berurutan
10	menang	boolean	Menandai status kemenangan pemain; diset false jika HP Pahlawan habis sebelum semua Monster dikalahkan

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1	serang() [AksiBertarung]	Abstract / Interface	Kontrak method serangan yang wajib diimplementasikan oleh setiap class yang menggunakan interface AksiBertarung
2	bertahan() [AksiBertarung]	Abstract / Interface	Kontrak method bertahan yang wajib diimplementasikan oleh setiap class yang menggunakan interface AksiBertarung
3	gunakanItem() [AksiBertarung]	Abstract / Interface	Kontrak method penggunaan item yang wajib diimplementasikan oleh setiap class yang menggunakan interface AksiBertarung
4	Karakter(String, int, int)	Constructor	Menginisialisasi atribut nama, hp, baseDamage, dan mengatur isDefending ke false sebagai nilai awal
5	getNama()	Functional (getter)	Mengembalikan nilai nama karakter
6	setNama(String)	Procedural (setter)	Mengatur nilai nama karakter
7	getHp()	Functional (getter)	Mengembalikan nilai hp karakter
8	setHp(int)	Procedural (setter)	Mengatur nilai hp karakter
9	getBaseDamage()	Functional (getter)	Mengembalikan nilai baseDamage karakter
10	setBaseDamage(int)	Procedural (setter)	Mengatur nilai baseDamage karakter
11	getIsDefending()	Functional (getter)	Mengembalikan status isDefending karakter
12	setIsDefending(boolean)	Procedural (setter)	Mengatur status isDefending karakter
13	terimaDamage(int)	Procedural	Mengurangi HP sesuai damage; jika isDefending aktif damage dipotong 50% lalu flag direset ke false
14	tampilkanStatus() [abstract]	Abstract	Kontrak abstract method untuk menampilkan status karakter; wajib di-override oleh setiap subclass
15	Pahlawan(String, int, int, int, int)	Constructor	Membuat objek Pahlawan dengan memanggil super() lalu menginisialisasi mana dan level
16	getMana()	Functional (getter)	Mengembalikan nilai mana Pahlawan
17	setMana(int)	Procedural (setter)	Mengatur nilai mana Pahlawan
18	getLevel()	Functional (getter)	Mengembalikan nilai level Pahlawan

19	setLevel(int)	Procedural (setter)	Mengatur nilai level Pahlawan
20	serang() [Override - Pahlawan]	Functional	Menghitung damage serangan fisik Pahlawan: baseDamage dikali level
21	serang(String, int) [Overload - Pahlawan]	Functional	Overloading: jika mana mencukupi konsumsi mana dan kembalikan baseDamage x level x 2; jika tidak kembalikan 0
22	bertahan() [Override - Pahlawan]	Procedural	Mengaktifkan mode bertahan dengan mengatur isDefending ke true dan mencetak pesan ke konsol
23	gunakanItem() [Override - Pahlawan]	Procedural	Memulihkan HP Pahlawan sebesar 30 poin menggunakan potion
24	tampilkanStatus() [Override - Pahlawan]	Procedural	Menampilkan status lengkap Pahlawan: nama, HP, mana, dan level ke konsol
25	Monster(String, int, int, String)	Constructor	Membuat objek Monster dengan memanggil super() lalu menginisialisasi jenisMonster
26	serang() [Override - Monster]	Functional	Mengembalikan nilai baseDamage sebagai damage serangan Monster tanpa pengali
27	bertahan() [Override - Monster]	Procedural	Monster memulihkan HP sebesar 10 poin dan mencetak pesan pemulihan ke konsol
28	gunakanItem() [Override - Monster]	Procedural	Monster menggunakan item untuk memulihkan HP sebesar 20 poin
29	tampilkanStatus() [Override - Monster]	Procedural	Menampilkan status Monster: nama, jenis, dan HP ke konsol

IV. Dokumentasi dan Pembahasan Code

Berikut adalah kode lengkap program beserta penjelasan tiap bagiannya:

AksiBertarung.java

```
public interface AksiBertarung {  
    int serang();  
    void bertahan();  
    void gunakanItem();  
}
```

Karakter.java

```
public abstract class Karakter implements AksiBertarung {  
    protected String nama;  
    protected int hp, baseDamage;  
    protected boolean isDefending;  
  
    public Karakter(String nama, int hp, int baseDamage) {  
        this.nama = nama;  
        this.hp = hp;  
        this.baseDamage = baseDamage;  
        isDefending = false;  
    }  
  
    public String getNama() {  
        return nama;  
    }  
  
    public void setNama(String nama) {  
        this.nama = nama;  
    }  
  
    public int getHp() {  
        return hp;  
    }  
  
    public void setHp(int hp) {  
        this.hp = hp;  
    }  
  
    public int getBaseDamage() {  
        return baseDamage;  
    }  
  
    public void setBaseDamage(int baseDamage) {  
        this.baseDamage = baseDamage;  
    }  
  
    public boolean getIsDefending() {  
        return isDefending;  
    }  
}
```

```

public void setIsDefending(boolean isDefending) {
    this.isDefending = isDefending;
}

public void terimaDamage(int damage) {
    if(isDefending) {
        damage = damage / 2;
        isDefending = false;
    }
    hp -= damage;
    if(hp < 0) {
        hp = 0;
    }
}

public abstract void tampilkanStatus();
}

```

Pahlawan.java

```

public class Pahlawan extends Karakter {
    private int mana, level;

    public Pahlawan(String nama, int hp, int baseDamage, int mana, int level) {
        super(nama, hp, baseDamage);
        this.mana = mana;
        this.level = level;
    }

    public int getMana() {
        return mana;
    }

    public void setMana(int mana) {
        this.mana = mana;
    }

    public int getLevel() {
        return level;
    }

    public void setLevel(int level) {
        this.level = level;
    }

    @Override
    public int serang() {
        return baseDamage * level;
    }

    public int serang(String namaSkill, int manaCost) {
        if(mana >= manaCost) {

```

```

        mana -= manaCost;
        System.out.println(nama + " menggunakan skill " + namaSkill);
        return baseDamage * level * 2;
    }
    System.out.println("Mana tidak cukup");
    return 0;
}

@Override
public void bertahan() {
    isDefending = true;
    System.out.println(nama + " bersiaga menahan serangan");
}

@Override
public void gunakanItem() {
    hp += 30;
    System.out.println(nama + " menggunakan potion");
}

@Override
public void tampilkanStatus() {
    System.out.println("Pahlawan: " + nama);
    System.out.println("HP: " + hp);
    System.out.println("Mana: " + mana);
    System.out.println("Level: " + level);
}
}

```

Monster.java

```

public class Monster extends Karakter {
    private String jenisMonster;

    public Monster(String nama, int hp, int baseDamage, String jenisMonster) {
        super(nama, hp, baseDamage);
        this.jenisMonster = jenisMonster;
    }

    @Override
    public int serang() {
        return baseDamage;
    }

    @Override
    public void bertahan() {
        hp += 10;
        System.out.println(nama + " memulihkan HP");
    }

    @Override
    public void gunakanItem() {

```



```

        hp += 20;
        System.out.println(nama + " menggunakan item monster");
    }

    @Override
    public void tampilkanStatus() {
        System.out.println("Monster: " + nama);
        System.out.println("Jenis: " + jenisMonster);
        System.out.println("HP: " + hp);
    }
}

```

GameEngine.java

```

public class GameEngine {
    public static void main(String[] args) {
        java.util.Scanner input = new java.util.Scanner(System.in);

        System.out.print("Masukkan nama pahlawan : ");
        String nama = input.nextLine();
        Pahlawan hero = new Pahlawan(nama, 150, 20, 100, 2);

        Monster[] daftarMonster = {
            new Monster("Goblin", 80, 10, "Normal"),
            new Monster("Orc", 120, 15, "Brute"),
            new Monster("Dragon", 200, 25, "Boss")
        };

        boolean menang = true;
        for(int i = 0; i < daftarMonster.length; i++) {

            Monster monster = daftarMonster[i];
            System.out.println("\n---- Musuh Muncul ----");
            monster.tampilkanStatus();

            while(hero.getHp() > 0 && monster.getHp() > 0) {
                System.out.println("\n---- STATUS ----");
                hero.tampilkanStatus();
                monster.tampilkanStatus();

                System.out.println("\n1. Serang");
                System.out.println("2. Skill");
                System.out.println("3. Bertahan / Heal");

                System.out.print("Pilih aksi : ");
                int pilih = input.nextInt();
                int damageHero = 0;

                switch(pilih) {
                    case 1:
                        damageHero = hero.serang();
                        System.out.println(hero.getNama() + " menyerang monster");

```

```

        monster.terimaDamage(damageHero);
        break;

    case 2:
        damageHero = hero.serang("Fire Blast", 20);
        monster.terimaDamage(damageHero);
        break;

    case 3:
        hero.bertahan();
        hero.gunakanItem();
        break;

    default:
        System.out.println("Pilihan tidak valid");
    }

    if(monster.getHp() > 0) {
        int damageMonster = monster.serang();
        System.out.println(monster.getNama() + " menyerang balik");
        hero.terimaDamage(damageMonster);
    }
}

if(hero.getHp() <= 0) {
    menang = false;
    break;
}
System.out.println(monster.getNama() + " berhasil dikalahkan");
}

System.out.println("\n----- HASIL GAME -----");
if(menang && hero.getHp() > 0) {
    System.out.println(hero.getNama() + " berhasil menamatkan Dungeon");
} else {
    System.out.println("Game Over");
}
input.close();
}
}

```

1. Interface AksiBertarung

AksiBertarung adalah interface yang mendefinisikan kontrak tiga method wajib bagi semua karakter dalam game. Dengan mendeklarasikan `serang()`, `bertahan()`, dan `gunakanItem()` di sini, program menjamin bahwa setiap class yang mengimplementasikannya pasti memiliki ketiga kemampuan tersebut. Interface ini adalah pondasi polymorphism berbasis kontrak dalam program, karena memastikan setiap karakter berperilaku konsisten meski implementasinya bisa sangat berbeda antara Pahlawan dan Monster.

2. Abstract Class Karakter

Karakter adalah abstract class yang mengimplementasikan AksiBertarung secara parsial. Atribut nama, hp, baseDamage, dan isDefending dideklarasikan di sini agar dapat diwariskan oleh semua subclass. Method terimaDamage() diimplementasikan penuh di Karakter karena logikanya sama untuk semua karakter: jika isDefending aktif damage dipotong 50% lalu flag direset. Method tampilkanStatus() dideklarasikan sebagai abstract karena format tampilannya berbeda antara Pahlawan dan Monster, sehingga setiap subclass wajib mengimplementasikannya sendiri.

3. Class Pahlawan (Subclass)

Pahlawan mewarisi Karakter dan menambahkan atribut mana serta level. Polymorphism diterapkan melalui method overriding: serang() di-override menghasilkan damage baseDamage dikali level, bertahan() mengaktifkan isDefending, gunakanItem() memulihkan HP 30 poin, dan tampilkanStatus() menampilkan detail lengkap Pahlawan. Selain itu, Pahlawan juga menerapkan method overloading dengan menyediakan serang(String, int) sebagai versi skill: jika mana mencukupi, skill diaktifkan dan damage menjadi dua kali lipat; jika tidak, method mengembalikan 0.

4. Class Monster (Subclass)

Monster juga mewarisi Karakter dan mengimplementasikan semua method yang di-override dengan perilaku berbeda dari Pahlawan. serang() hanya mengembalikan baseDamage tanpa pengali level, mencerminkan kesederhanaan AI Monster. bertahan() memulihkan HP 10 poin dan gunakanItem() memulihkan HP 20 poin sebagai mekanisme survival Monster. tampilkanStatus() menampilkan nama, jenis, dan HP Monster. Atribut jenisMonster yang eksklusif di class ini memungkinkan diferensiasi tipe Monster: Normal, Brute, dan Boss.

5. Class GameEngine (Main) dan Polymorphism

GameEngine adalah entry point yang mengorkestrasi seluruh alur permainan. Pahlawan dibuat dengan HP 150, damage 20, mana 100, dan level 2. Tiga Monster disimpan dalam array Monster[] dan dihadapi satu per satu dalam loop for. Di dalam while loop pertarungan, pemain memilih aksi melalui switch-case: pilihan 1 memanggil serang() fisik Pahlawan, pilihan 2 memanggil overload serang() dengan skill Fire Blast berbiaya 20 mana, dan pilihan 3 memanggil bertahan() lalu gunakanItem(). Setelah aksi pemain, Monster menyerang balik jika HP-nya masih di atas 0. Polymorphism runtime terlihat jelas di sini karena setiap pemanggilan method serang(), bertahan(), dan gunakanItem() akan mengeksekusi implementasi yang sesuai dengan tipe objek sesungguhnya di runtime.

V. Kesimpulan

Polymorphism dalam program ini diterapkan melalui dua mekanisme utama. Pertama, method overriding: subclass Pahlawan dan Monster masing-masing mengimplementasikan ulang method `serang()`, `bertahan()`, `gunakanItem()`, dan `tampilkanStatus()` dari abstract class `Karakter` dan interface `AksiBertarung` sesuai logika dan perilakunya sendiri. Hal ini memungkinkan satu nama method menghasilkan perilaku yang berbeda tergantung pada tipe objek yang memanggilnya, sehingga kode di `GameEngine` menjadi lebih bersih dan seragam meski mengelola dua jenis karakter yang berbeda.

Kedua, method overloading pada class `Pahlawan` memungkinkan satu nama method `serang()` melayani dua skenario yang berbeda: serangan fisik biasa tanpa parameter dan serangan skill dengan parameter nama skill serta biaya mana. Penggunaan interface `AksiBertarung` dan abstract class `Karakter` juga menunjukkan penerapan prinsip abstraksi yang baik karena kontrak perilaku ditetapkan di level yang lebih tinggi, sementara detail implementasinya diserahkan sepenuhnya kepada masing-masing subclass. Dengan desain ini, program menjadi lebih modular dan mudah dikembangkan karena penambahan jenis karakter baru cukup dilakukan dengan membuat subclass baru dari `Karakter` tanpa mengubah kode `GameEngine` yang sudah ada.